King Abdulaziz University

Faculty of Computing and Information Technology

Slate

An Appointment-Scheduling Web Application

CPIS-358 — Internet Application and Web Programming

Instructor: Dr. Fouad Alallah · Group 10

Team Member	Student ID
Abdullah Ahmed Alamoudi	2236724
Ibrahim Turki Alothman	2136922

Table of Contents

1. Overview

Slate is a web application for booking and managing appointments. A visitor can browse the available services and prices, create an account, and book a time slot; from a personal dashboard they can then search, filter, reschedule, confirm, cancel, or delete their appointments.

The project was originally built for CPIS-358 as an ASP.NET MVC site across three assignments. This document describes a **complete rebuild** that keeps the original idea but fixes its data model, security, and structure, and delivers a clean REST API with a matching front-end.

Objectives

- Let users **register and sign in** securely.
- Let users **book appointments** against a catalogue of services.
- Give each user a **dashboard** to manage their own appointments.
- Expose all behaviour through a **documented REST API** with server-side validation.

2. Architecture

The application is a small client–server system. A Node.js/Express server exposes a JSON REST API and also serves the static front-end. Data is stored in SQLite through Node's built-in `node:sqlite` module, so there is no native database to install.

Layer	Technology & responsibility
Front-end	Static HTML, CSS, and vanilla JavaScript. Talks to the API with fetch().
API server	Express. Routing, validation, authentication, business rules.
Auth	bcryptjs hashes passwords; jsonwebtoken issues signed session tokens (JWT).
Storage	SQLite via node:sqlite. Three tables with foreign keys and constraints.

Request flow

The browser stores a JWT after login and sends it as a `Bearer` token on each protected request. The server verifies the token, loads the user, and scopes every query to that user — so a person can only ever read or change their own appointments.

3. Data Model

Three tables, with integer primary keys and proper foreign keys. A uniqueness constraint prevents a user from holding two appointments in the same slot.

```
users         (id PK, name, email UNIQUE, password_hash, created_at)
services      (id PK, name UNIQUE, description, price, duration_min)
appointments  (id PK,
               user_id     FK -> users(id)    ON DELETE CASCADE,
               service_id  FK -> services(id),
               date, time, status, notes, created_at,
               UNIQUE (user_id, date, time))
```

`status` is constrained to `pending`, `confirmed`, or `cancelled`. Dates are stored as `YYYY-MM-DD` and times as `HH:MM`.

4. REST API

All endpoints return JSON. Routes under appointments and stats require an `Authorization: Bearer <token>` header.

Method	Path	Description
POST	`/api/auth/register`	Create an account; returns a token and user.
POST	`/api/auth/login`	Authenticate; returns a token and user.
GET	`/api/auth/me`	Return the current user.
GET	`/api/services`	List services and prices (public).
GET	`/api/stats`	Average service price and counts by status.
GET	`/api/appointments`	List own appointments; filters: status, q, from, to.
POST	`/api/appointments`	Create an appointment.
GET	`/api/appointments/:id`	Get one appointment (own).
PUT	`/api/appointments/:id`	Update an appointment (own).
PATCH	`/api/appointments/:id/status`	Change status only.
DELETE	`/api/appointments/:id`	Delete an appointment (own).

Validation rules

- **Email** must match a standard email pattern; duplicates are rejected at registration.
- **Password** must be at least 8 characters with an uppercase letter, a lowercase letter, and a digit.
- **Bookings** cannot be in the past, and a slot already taken by the same user returns a `409` conflict.
- All queries are **parameterised**, so user input cannot alter the SQL.

5. Front-End

The interface is built from static pages sharing one design system (teal and amber on a cool off-white, with Fraunces for headings and Inter for body text). A small shared script handles API calls, stores the session token, and renders the navigation based on whether the user is signed in.

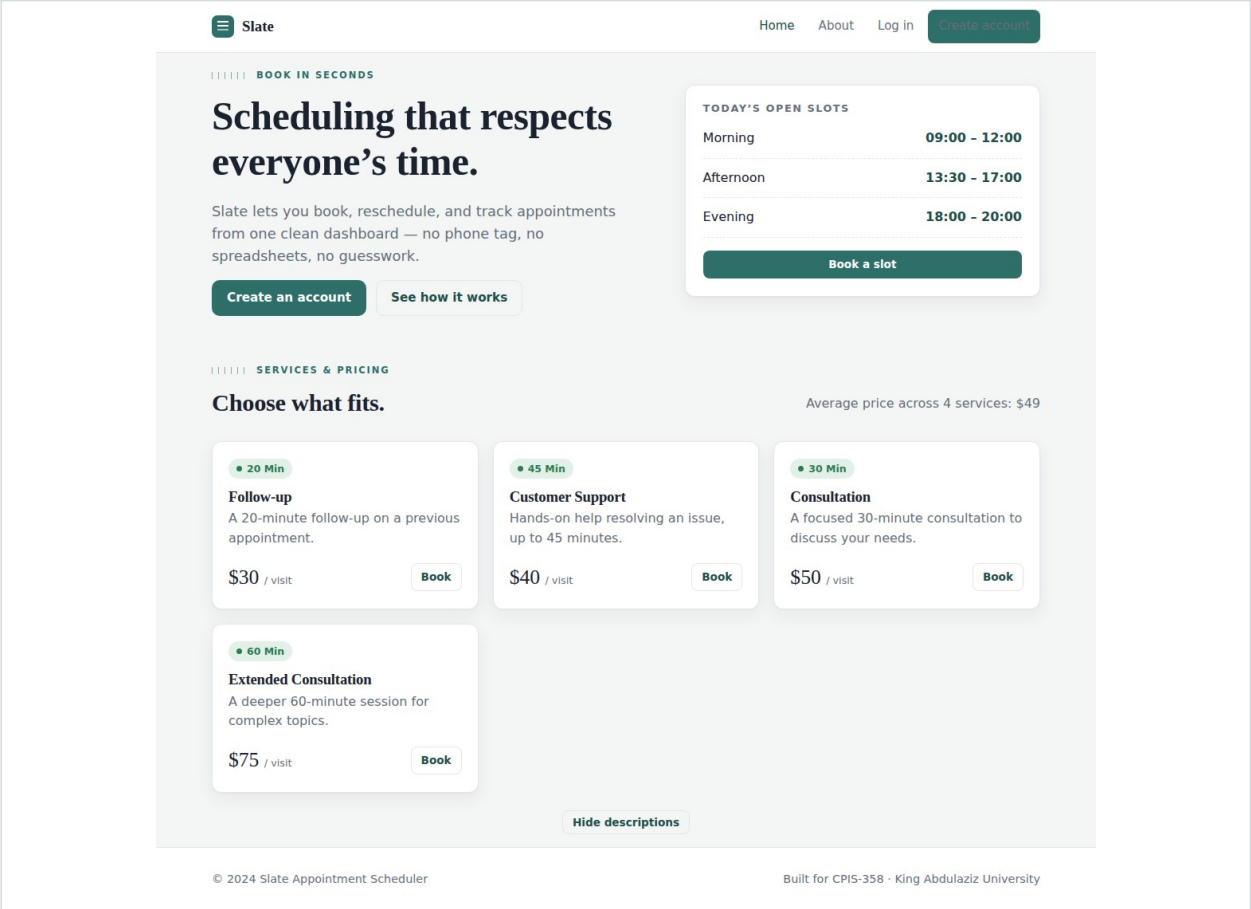

Figure 1 — Home page: services and live average price.

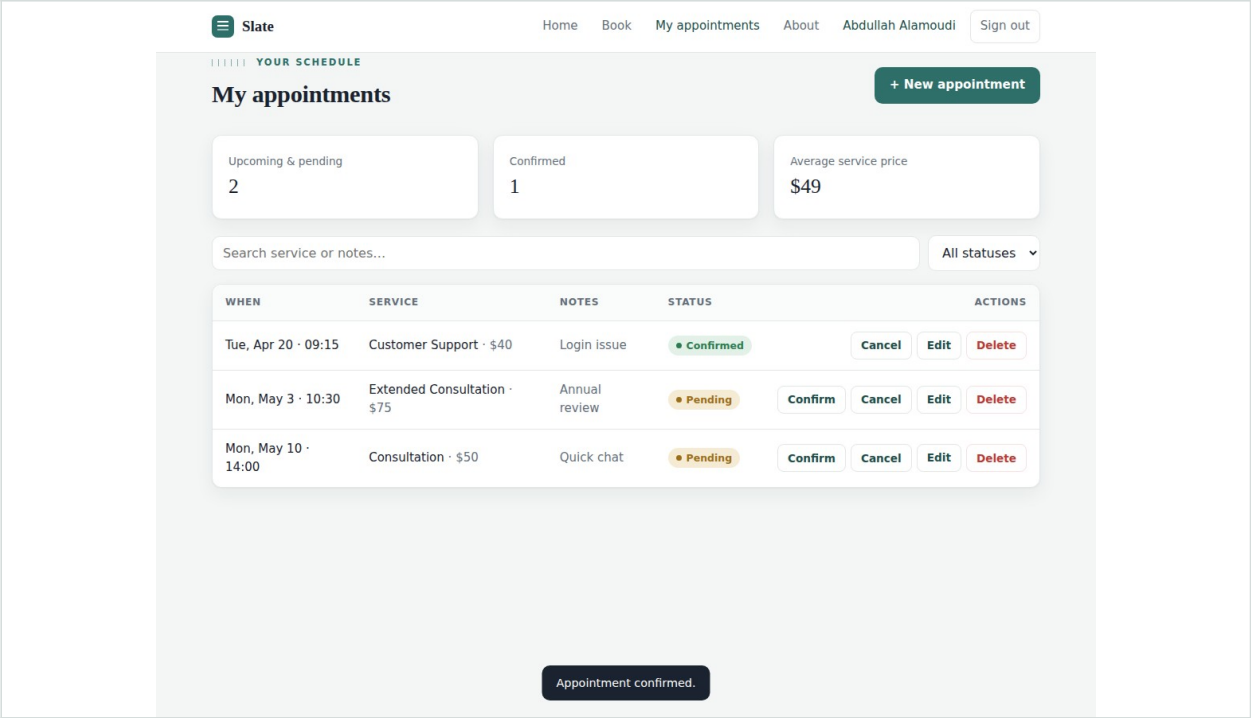

Figure 2 — Dashboard: search, filter, status pills, and per-row actions.

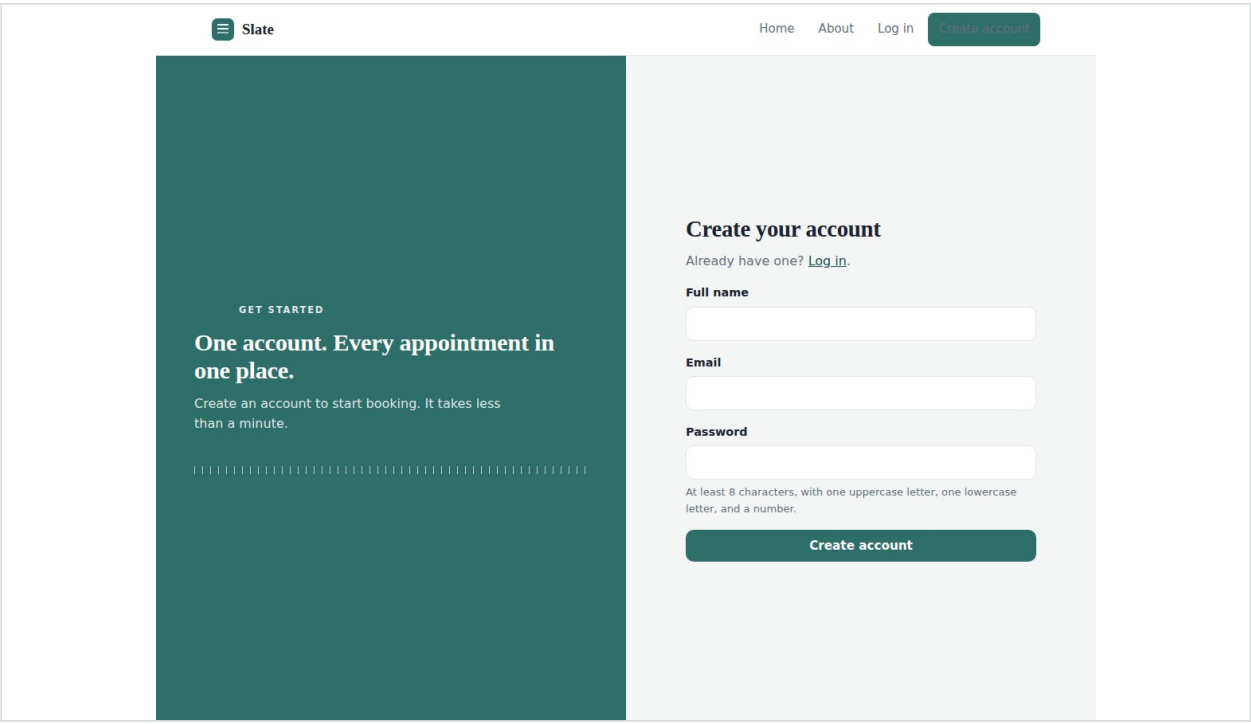

Figure 3 — Account creation, with the same validation the server enforces.

6. Problems Fixed from the Original

The original ASP.NET version had several correctness, security, and design problems. Each was addressed in the rebuild:

Broken data model

- The user model used an integer **primary key named `date`** and stored the email in a field named `time`. → Replaced with a clear `users` table.
- Appointments used the appointment **type as the primary key**, so only one appointment per type could exist, and none were linked to a user. → Replaced with an `id`-keyed table tied to a user, service, date, time, and status.

Security

- **Passwords were stored in plain text.** → Now hashed with bcrypt.
- **Login did no real authentication** — it only checked for non-empty fields and redirected. → Now verifies the password hash and issues a JWT.
- A leftover search action **echoed user input straight back** into the response. → Replaced with parameterised search over the user's own data.

Structure & front-end

- `Home` and `Account` controllers **duplicated each other** with two parallel login/scheduling flows. → Consolidated into one API.
- Views shipped **broken asset paths and missing images**, duplicated inline styles, and full HTML documents inside a shared layout. → Rebuilt as one consistent design system.

7. Running the Project

```
npm install
npm start
# open http://localhost:3000
```

The SQLite database is created and seeded with the default services on first run. Configuration is via optional environment variables: `PORT`, `JWT_SECRET`, and `DB_PATH`. Requires Node.js 22.5 or newer (for the built-in SQLite module).

Repository layout

```
server/server.js   API + all routes
server/db.js       SQLite schema + seed
public/            HTML, CSS, JS front-end
README.md          setup + API reference
```